

STOR 664 — Part 1 Combined EDA (Pokémon)

Team: Yinyu Yao, Kareena Legare, Samuel Moore, Irene Zhang

2025-11-15

Introduction

We used the Pokemon dataset from TidyTuesday. [Dataset Link](#)

Pokemon is a Japanese video game where you catch Pokemon in the wild and use them to fight battles against other Pokemon. Pokemon are wild monsters and each one has a type, like fire, water, grass, etc. The type of a Pokemon will affect battle outcomes through a system of strengths and weakness. For example, fire attacks are very effective against grass Pokemon which results in increased damage. Pokemon also have basic stats like HP (Hit Points), attack, defense etc. These stats will also influence a Pokemon's performance. The stat we are most interested in is HP which determines how much damage a Pokemon can take before fainting. The dataset we chose has 949 rows and 22 columns with information about the Pokemon's name, height weight, stats, type etc.

The question we want to answer is: What characteristics of Pokemon best predict their HP stat? Is there a relationship between HP and the Pokemon's other characteristics?

We want to answer these questions because it will provide insight into how Pokemon are designed and if there are any patterns in determining a Pokemon's stats. For example, perhaps Pokemon with higher attack or speed might have lower HP indicating that these Pokemon are designed to do high damage quickly. Or perhaps Pokemon with higher defense have higher HP indicating that these Pokemon are designed to last longer and endure more damage. Understanding the relationship between HP and other parameters will allow players to gain a deeper understanding of the game and more effectively strategize their battles.

There are many existing projects that explore a Pokemon dataset. These can be found on Kaggle and Medium. Many of them do data visualization or EDA. In particular, this [Project](#) attempts to answer a similar question as ours and performed a simple linear regression with HP as their Y-variable.

EDA

We first did extensive EDA on our dataset.

```
# 1) Dataset Summary and Statistics

# NA counts across ALL columns (1-row audit table)
na_counts <- pokemon |>
  summarise(across(everything(), ~sum(is.na(.)))) |>
  tidyr::pivot_longer(
    everything(),
    names_to = "variable",
    values_to = "na_count"
  ) |>
  dplyr::filter(na_count > 0) |>
```

```

  arrange(desc(na_count))
knitr::kable(na_counts, digits = 0,
             caption = "Columns with at least one missing value.")

```

Table 1: Columns with at least one missing value.

variable	na_count
egg_group_2	711
type_2	439
generation_id	147

```

# Type1 / Generation counts
gen_tab <- as.data.frame(table(pokemon$generation_id, useNA = "ifany"))
names(gen_tab) <- c("generation_id", "count")
type_tab <- as.data.frame(table(pokemon$type_1, useNA = "ifany"))
names(type_tab) <- c("type_1", "count")

knitr::kable(gen_tab, caption = "Counts of Pokémon by generation_id")

```

Table 2: Counts of Pokémon by generation_id

generation_id	count
1	151
2	100
3	135
4	107
5	156
6	72
7	81
NA	147

```

knitr::kable(type_tab, caption = "Counts of Pokémon by type_1")

```

Table 3: Counts of Pokémon by type_1

type_1	count
bug	79
dark	37
dragon	39
electric	61
fairy	19
fighting	31
fire	59
flying	4
ghost	40
grass	84
ground	36
ice	29

type_1	count
normal	111
poison	35
psychic	64
rock	65
steel	30
water	126

```
# egg_group_1 counts
egg1_counts <- pokemon |> count(egg_group_1) |> arrange(desc(n))
knitr::kable(egg1_counts, caption = "Counts of egg_group_1")
```

Table 4: Counts of egg_group_1

egg_group_1	n
ground	218
no-eggs	118
monster	94
water1	84
bug	80
mineral	67
indeterminate	64
flying	59
humanshape	40
fairy	38
plant	35
water2	21
water3	16
dragon	14
ditto	1

```
# 2) Conditional distribution P(type_2 | type_1) + X^2 ----

pokemon_types <- pokemon |> filter(!is.na(type_2))

type_pair_counts <- pokemon_types |>
count(type_1, type_2, name = "n") |>
group_by(type_1) |>
mutate(row_total = sum(n), prop = n / row_total) |>
arrange(type_1, desc(prop)) |>
ungroup()

knitr::kable(head(type_pair_counts, 15), digits = 3,
caption = "P(type_2 | type_1): top 15 rows.")
```

Table 5: P(type_2 | type_1): top 15 rows.

type_1	type_2	n	row_total	prop
bug	flying	14	61	0.230

type_1	type_2	n	row_total	prop
bug	poison	12	61	0.197
bug	steel	7	61	0.115
bug	grass	6	61	0.098
bug	electric	5	61	0.082
bug	fighting	4	61	0.066
bug	rock	3	61	0.049
bug	water	3	61	0.049
bug	fairy	2	61	0.033
bug	fire	2	61	0.033
bug	ground	2	61	0.033
bug	ghost	1	61	0.016
dark	flying	5	25	0.200
dark	dragon	4	25	0.160
dark	fire	3	25	0.120

```

type2_pref <- type_pair_counts |>
group_by(type_1) |>
slice_max(prop, n = 1, with_ties = TRUE) |>
arrange(type_1, desc(prop)) |>
ungroup()

knitr::kable(type2_pref, digits = 3,
caption = "Most frequent secondary type(s) for each primary type.")

```

Table 6: Most frequent secondary type(s) for each primary type.

type_1	type_2	n	row_total	prop
bug	flying	14	61	0.230
dark	flying	5	25	0.200
dragon	ground	8	27	0.296
electric	flying	6	21	0.286
fairy	flying	2	2	1.000
fighting	psychic	3	9	0.333
fire	fighting	7	28	0.250
fire	flying	7	28	0.250
flying	dragon	2	2	1.000
ghost	grass	11	30	0.367
grass	poison	15	45	0.333
ground	flying	4	21	0.190
ice	ground	3	14	0.214
ice	water	3	14	0.214
normal	flying	27	44	0.614
poison	dark	5	20	0.250
psychic	fairy	7	23	0.304
psychic	flying	7	23	0.304
rock	flying	18	53	0.340
steel	psychic	7	25	0.280
water	ground	10	60	0.167

```

tab_t1_t2 <- table(type_1 = pokemon_types$type_1, type_2 = pokemon_types$type_2)
chi_res <- chisq.test(tab_t1_t2) # prints Pearson  $X^2$ 

## Warning in stats::chisq.test(x, y, ...): Chi-squared approximation may be
## incorrect

chi_tbl <- tibble(statistic = as.numeric(chi_res$statistic),
df           = as.integer(chi_res$parameter),
p_value      = as.numeric(chi_res$p.value))
knitr::kable(chi_tbl, caption = "Chi-squared test of independence between type_1 and type_2.")

```

Table 7: Chi-squared test of independence between type_1 and type_2.

statistic	df	p_value
698.6441	289	0

```
chi_res
```

```

##
## Pearson's Chi-squared test
##
## data:  tab_t1_t2
## X-squared = 698.64, df = 289, p-value < 2.2e-16

```

3) By generation_id: distribution tables and boxplots

```

pokemon <- pokemon |>
mutate(generation_id_f = forcats::fct_na_value_to_level(as.factor(generation_id), level = "Unknown"))
stat_vars <- c("hp", "attack", "defense", "special_attack", "special_defense", "speed")

gen_stats_all <- pokemon |>
  group_by(generation_id_f) |>
  summarise(
    n = n(),
    across(all_of(stat_vars),
      list(mean = ~mean(.x, na.rm = TRUE),
           sd   = ~sd(.x,   na.rm = TRUE)),
      .names = "{.col}_{.fn}")
  )

gen_stats_long <- gen_stats_all |>
  tidyr::pivot_longer(
    cols = -c(generation_id_f, n),
    names_to = c("stat", "metric"),
    names_sep = "_(?=[^_]+$)"
  ) |>
  tidyr::pivot_wider(
    names_from = metric,
    values_from = value
  )

```

```

) |>
# ensure stat is ordered like stat_vars: hp, attack, defense, ...
mutate(stat = factor(stat, levels = stat_vars)) |>
arrange(stat, generation_id_f)

knitr::kable(
  gen_stats_long,
  digits = 1,
  caption = "Means and SDs of stats by generation (including Unknown)."
```

Table 8: Means and SDs of stats by generation (including Unknown).

generation_id_f	n	stat	mean	sd
1	151	hp	64.2	28.6
2	100	hp	71.0	31.2
3	135	hp	65.7	25.2
4	107	hp	73.1	24.7
5	156	hp	70.3	21.6
6	72	hp	68.9	21.7
7	81	hp	70.7	28.2
Unknown	147	hp	70.1	25.1
1	151	attack	72.9	26.8
2	100	attack	68.3	28.4
3	135	attack	73.1	30.4
4	107	attack	80.2	30.9
5	156	attack	81.0	29.4
6	72	attack	72.5	25.6
7	81	attack	83.2	32.6
Unknown	147	attack	98.9	37.5
1	151	defense	68.2	26.9
2	100	defense	69.7	35.2
3	135	defense	69.0	31.1
4	107	defense	75.2	30.7
5	156	defense	71.2	23.0
6	72	defense	75.2	31.7
7	81	defense	77.0	29.9
Unknown	147	defense	87.7	34.4
1	151	special_attack	67.1	28.5
2	100	special_attack	64.5	25.6
3	135	special_attack	67.9	28.3
4	107	special_attack	73.3	31.2
5	156	special_attack	69.2	29.8
6	72	special_attack	72.5	28.0
7	81	special_attack	73.5	33.4
Unknown	147	special_attack	92.0	42.0
1	151	special_defense	66.1	24.2
2	100	special_defense	72.3	31.5
3	135	special_defense	66.5	28.5
4	107	special_defense	74.5	27.8
5	156	special_defense	67.3	21.9

generation_id_f	n	stat	mean	sd
6	72	special_defense	74.7	30.8
7	81	special_defense	74.8	29.3
Unknown	147	special_defense	84.6	26.2
1	151	speed	69.1	27.0
2	100	speed	61.4	27.2
3	135	speed	61.6	26.9
4	107	speed	69.5	27.6
5	156	speed	66.6	28.2
6	72	speed	65.7	25.9
7	81	speed	64.5	29.1
Unknown	147	speed	87.3	31.4

```

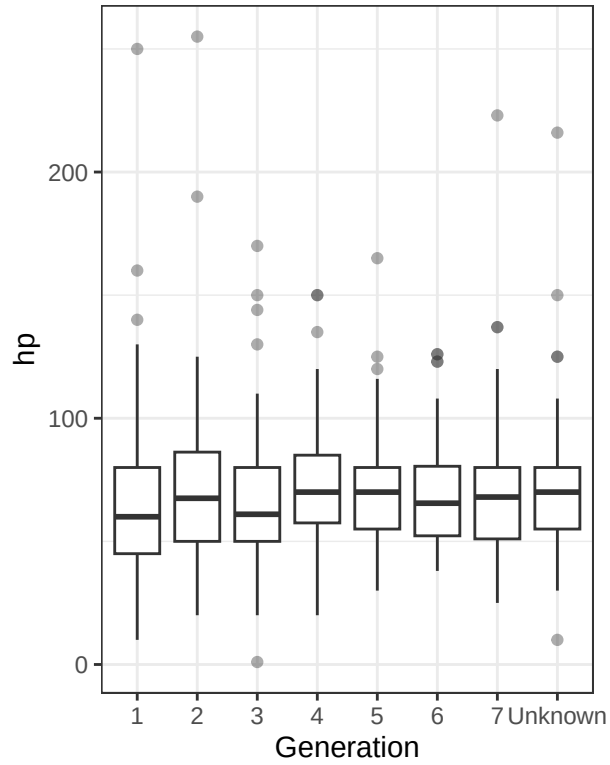
gen_plots <- lapply(stat_vars, function(s) {
  ggplot(pokemon, aes(x = generation_id_f, y = .data[[s]])) +
    geom_boxplot(outlier.alpha = 0.4) +
    labs(
      title = paste("Dist. of", s, "by generation"),
      x = "Generation",
      y = s
    ) +
    theme_bw()
})
names(gen_plots) <- stat_vars

gen_panel_1 <- gen_plots[["hp"]] + gen_plots[["attack"]]
gen_panel_2 <- gen_plots[["defense"]] + gen_plots[["special_attack"]]
gen_panel_3 <- gen_plots[["special_defense"]] + gen_plots[["speed"]]

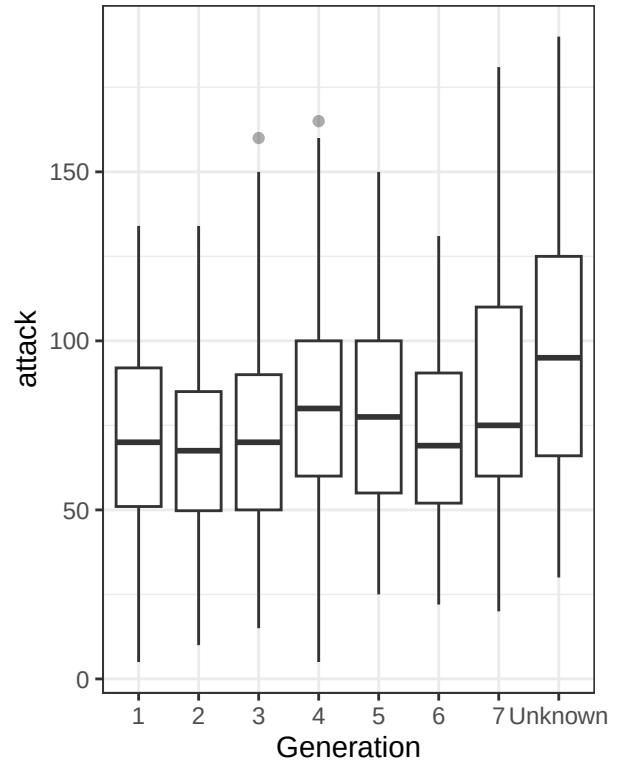
gen_panel_1

```

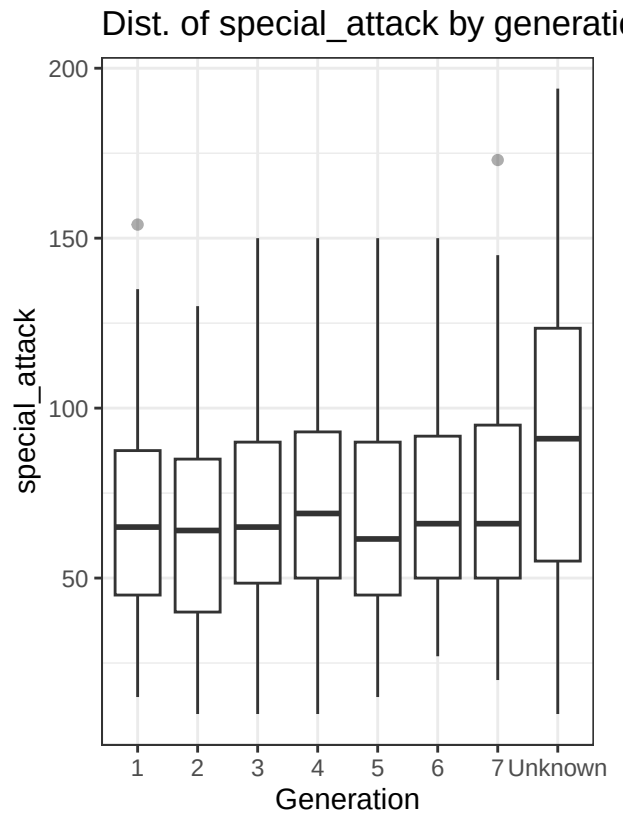
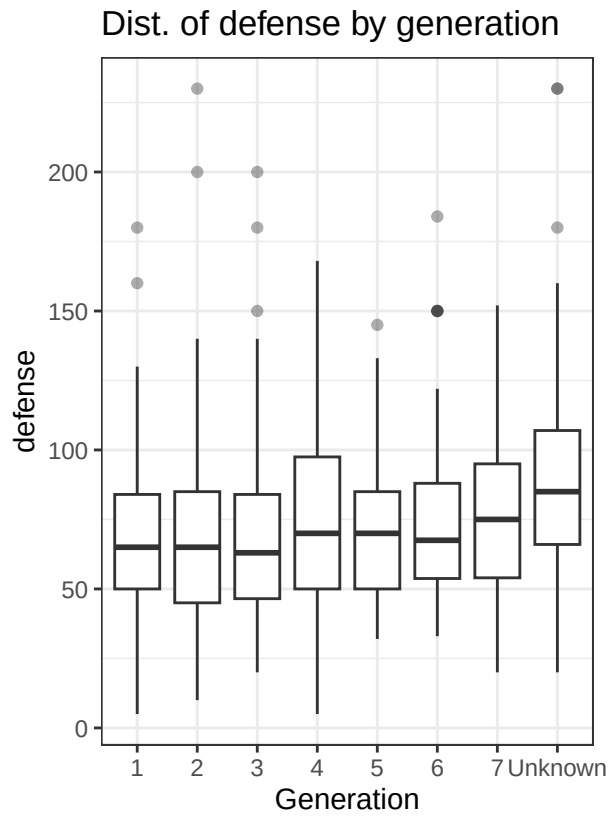
Dist. of hp by generation



Dist. of attack by generation

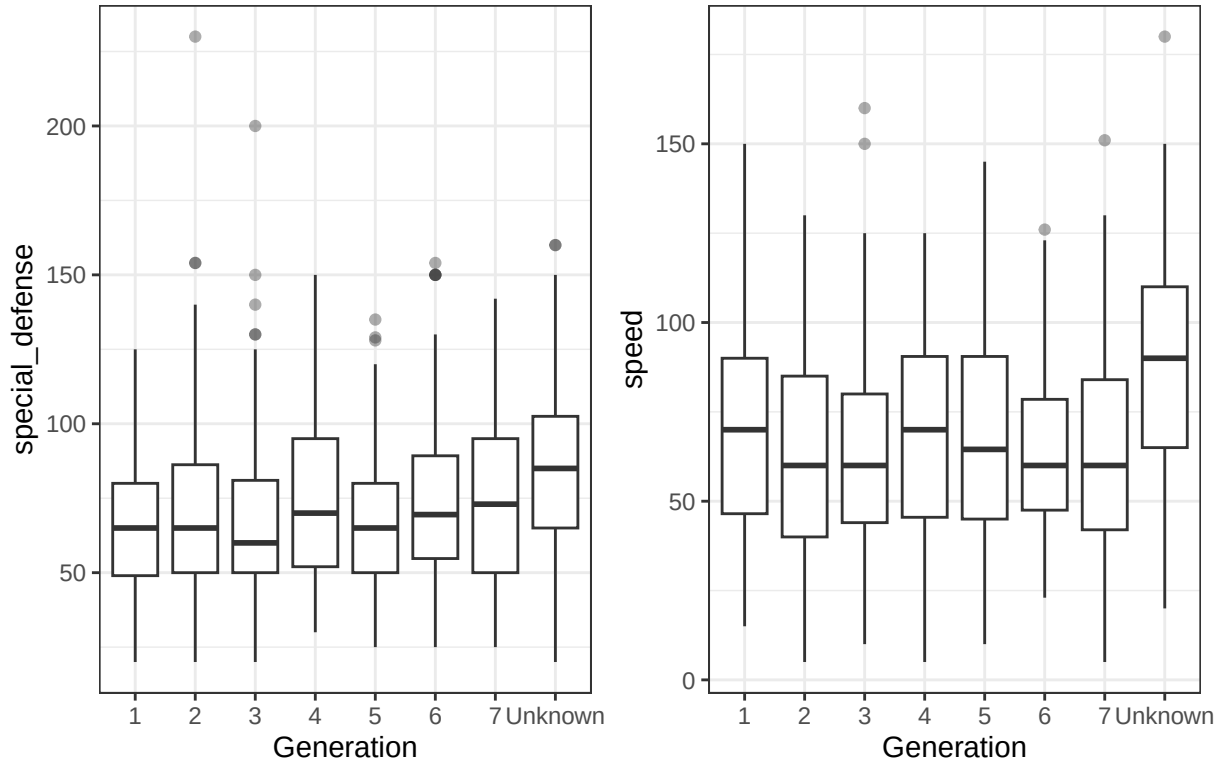


gen_panel_2



gen_panel_3

Dist. of special_defense by generation Dist. of speed by generation



4) By type1: distribution tables and boxplots

```
pokemon <- pokemon |>
  mutate(type_1_f = forcats::fct_infreq(as.factor(type_1)))
```

```
type1_stats <- pokemon |>
  group_by(type_1_f) |>
  summarise(
    n = n(),
    across(all_of(stat_vars),
      list(mean = ~mean(.x, na.rm = TRUE),
           sd   = ~sd(.x,   na.rm = TRUE)),
      .names = "{.col}_{.fn}")
  )
```

Table 1: HP and Speed

```
type1_hp_speed <- type1_stats |>
  select(type_1_f, n, hp_mean, hp_sd, speed_mean, speed_sd)
```

```
knitr::kable(
  type1_hp_speed,
  digits = 1,
  caption = "Means and SDs of HP and Speed by primary type (type_1)."
)
```

Table 9: Means and SDs of HP and Speed by primary type (type_1).

type_1_f	n	hp_mean	hp_sd	speed_mean	speed_sd
water	126	71.1	26.4	65.7	24.8
normal	111	77.3	34.8	70.3	27.7
grass	84	66.7	18.9	60.3	27.9
bug	79	57.7	17.3	63.3	34.0
rock	65	65.3	19.4	64.3	33.2
psychic	64	72.6	29.8	80.9	36.3
electric	61	55.8	18.3	87.2	24.0
fire	59	69.7	18.8	73.5	24.8
ghost	40	64.2	28.7	65.7	29.1
dragon	39	84.5	32.0	82.9	22.8
dark	37	69.7	33.2	76.6	26.8
ground	36	71.6	27.5	64.6	27.9
poison	35	67.2	19.6	64.2	25.7
fighting	31	71.6	25.7	67.6	26.9
steel	30	67.3	16.6	56.1	24.6
ice	29	70.5	20.8	64.6	24.2
fairy	19	72.9	22.9	53.6	26.6
flying	4	70.8	20.7	102.5	32.1

```
# Table 2: Attack and Special Attack
type1_atk_spa <- type1_stats |>
  select(type_1_f, n,
         attack_mean, attack_sd,
         special_attack_mean, special_attack_sd)

knitr::kable(
  type1_atk_spa,
  digits = 1,
  caption = "Means and SDs of Attack and Special Attack by primary type (type_1)."
```

Table 10: Means and SDs of Attack and Special Attack by primary type (type_1).

type_1_f	n	attack_mean	attack_sd	special_attack_mean	special_attack_sd
water	126	74.7	29.0	75.6	30.3
normal	111	75.9	30.0	57.5	25.1
grass	84	75.2	29.3	76.2	27.0
bug	79	72.5	37.2	57.7	31.0
rock	65	90.0	31.9	66.2	28.0
psychic	64	72.5	42.0	98.0	39.1
electric	61	68.1	22.4	83.0	32.6
fire	59	84.3	27.5	87.6	29.0
ghost	40	76.3	28.5	77.3	30.8
dragon	39	108.7	32.4	93.4	39.8
dark	37	84.7	26.3	71.4	32.8
ground	36	96.2	32.2	55.3	26.8

type_1_f	n	attack_mean	attack_sd	special_attack_mean	special_attack_sd
poison	35	73.5	19.8	62.5	21.7
fighting	31	99.1	28.0	53.8	27.3
steel	30	93.1	28.8	73.0	34.3
ice	29	73.1	27.3	72.3	29.1
fairy	19	61.2	28.1	81.2	28.9
flying	4	78.8	37.5	94.2	34.8

Table 3: Defense and Special Defense

```
type1_def_spdef <- type1_stats |>
  select(type_1_f, n,
         defense_mean, defense_sd,
         special_defense_mean, special_defense_sd)

knitr::kable(
  type1_def_spdef,
  digits = 1,
  caption = "Means and SDs of Defense and Special Defense by primary type (type_1)."
)
```

Table 11: Means and SDs of Defense and Special Defense by primary type (type_1).

type_1_f	n	defense_mean	defense_sd	special_defense_mean	special_defense_sd
water	126	73.5	28.1	72.4	29.6
normal	111	60.5	23.5	64.4	25.2
grass	84	71.7	25.0	70.7	22.1
bug	79	72.1	34.1	64.4	31.0
rock	65	94.3	34.4	75.3	30.2
psychic	64	69.9	29.1	87.0	31.1
electric	61	61.2	23.7	69.1	21.4
fire	59	69.3	25.0	71.9	22.0
ghost	40	82.0	29.6	78.7	25.5
dragon	39	89.3	25.0	88.3	28.4
dark	37	67.6	24.5	67.8	24.1
ground	36	82.6	33.5	62.9	20.7
poison	35	69.3	24.4	65.9	23.6
fighting	31	67.2	18.2	64.9	21.9
steel	30	124.8	42.7	83.6	29.0
ice	29	73.4	32.5	74.7	35.0
fairy	19	67.1	18.7	88.3	30.2
flying	4	66.2	21.4	72.5	22.2

```
type_plots <- lapply(stat_vars, function(s) {
  ggplot(pokemon, aes(x = type_1_f, y = .data[[s]])) +
    geom_boxplot(outlier.alpha = 0.4) +
    coord_flip() +
    labs(
      title = paste("Dist. of", s, "by type_1"),
      x = "Primary type (type_1)",
      y = s
    )
})
```

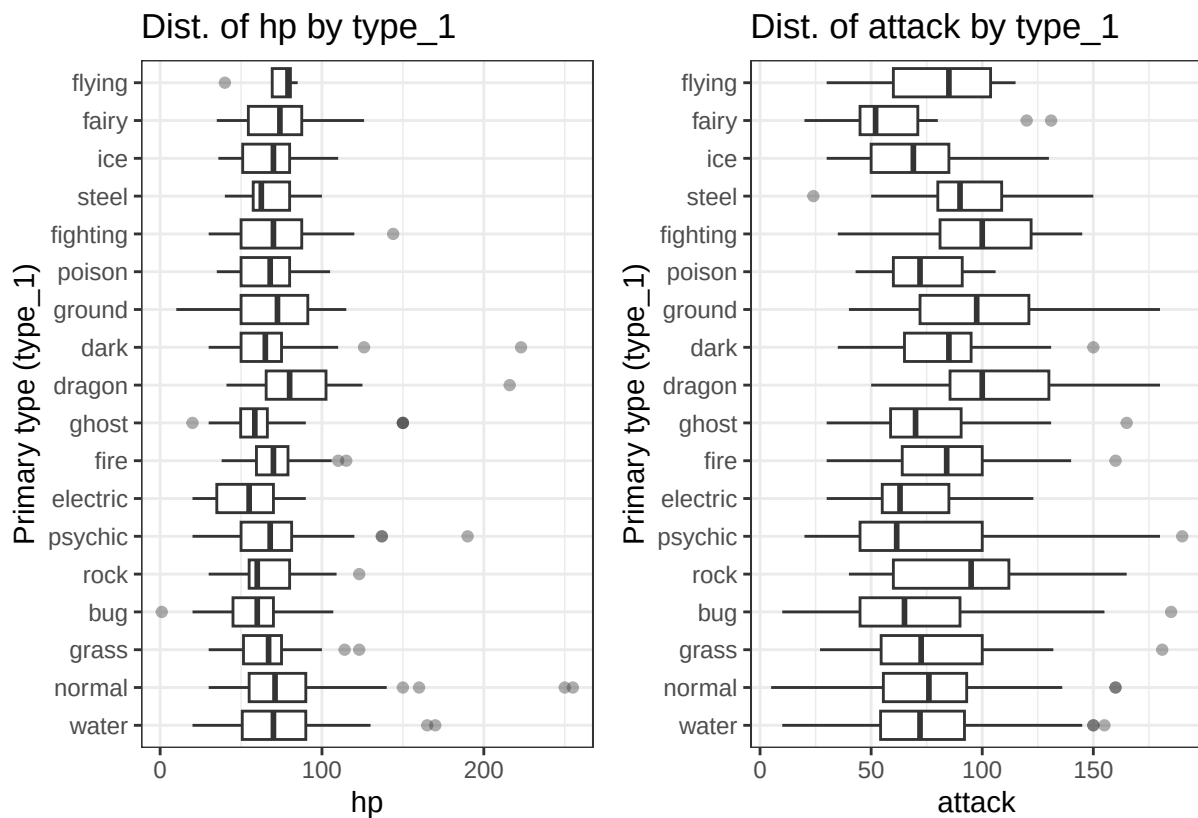
```

) +
  theme_bw()
})
names(type_plots) <- stat_vars

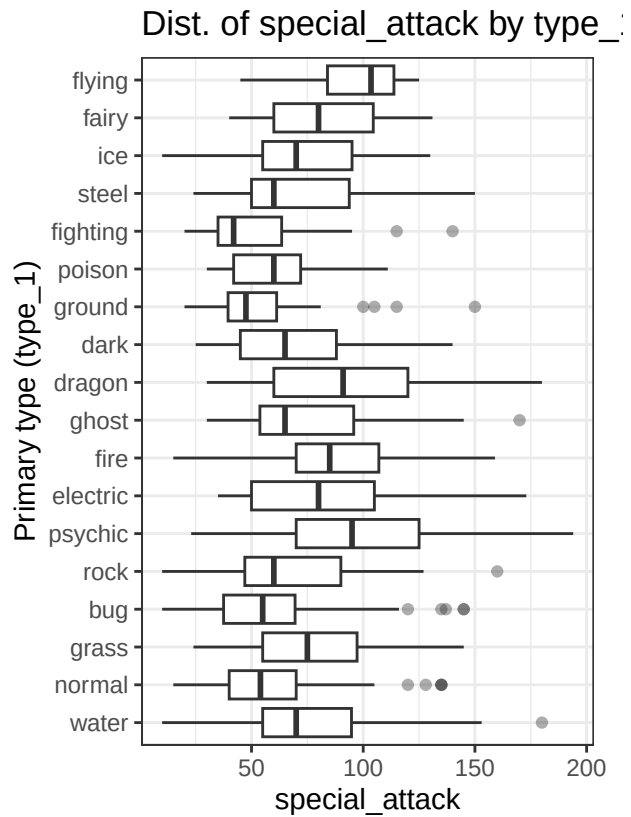
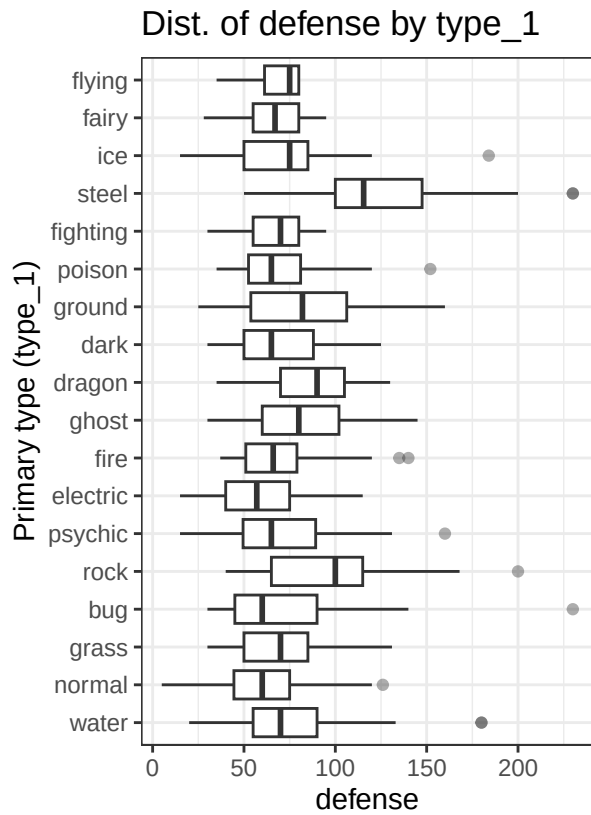
type_panel_1 <- type_plots[["hp"]]
type_panel_2 <- type_plots[["defense"]]
type_panel_3 <- type_plots[["special_defense"]] + type_plots[["speed"]]

type_panel_1

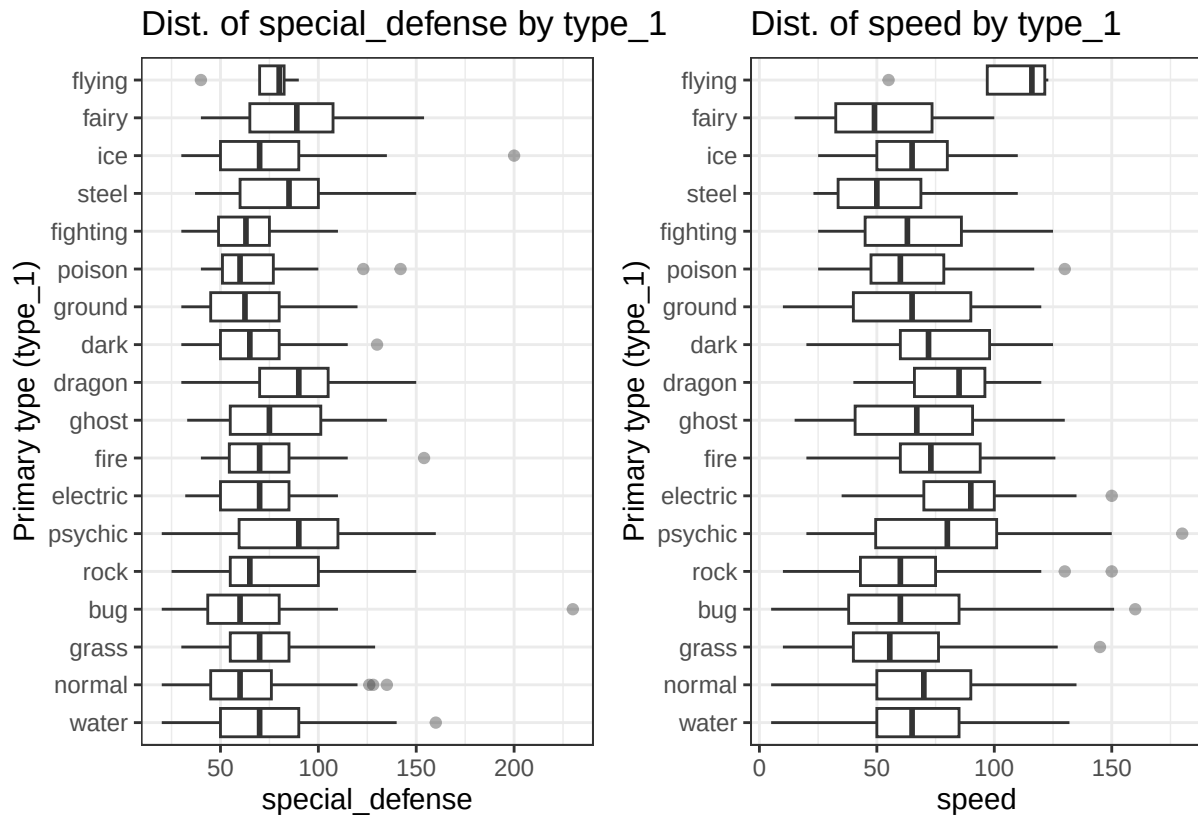
```



```
type_panel_2
```



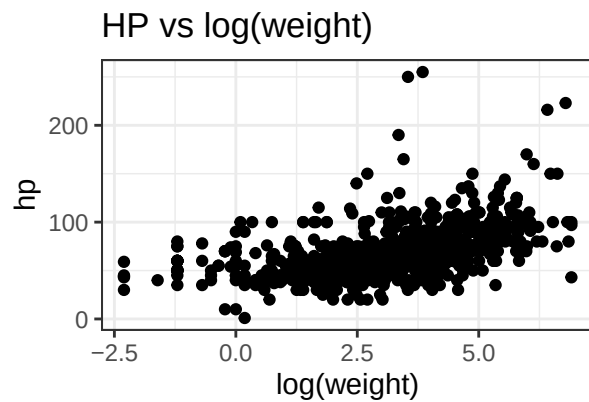
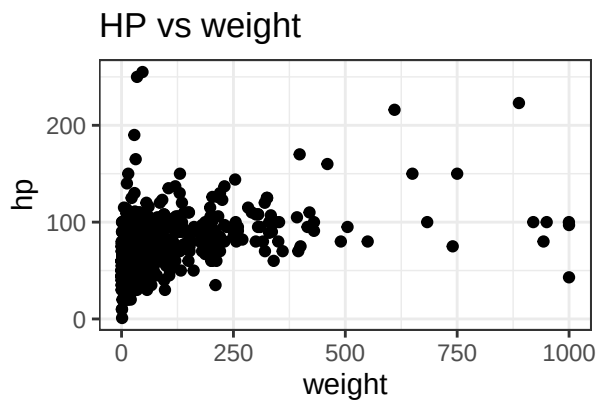
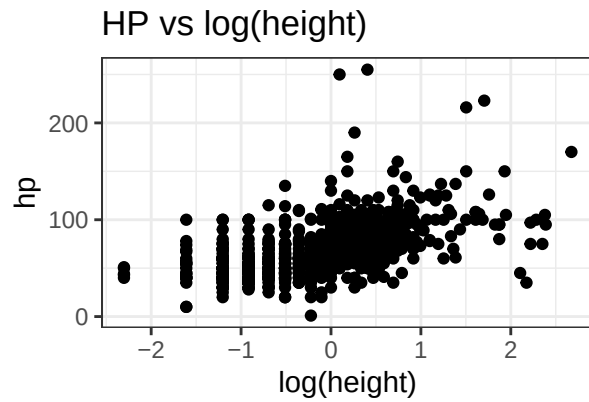
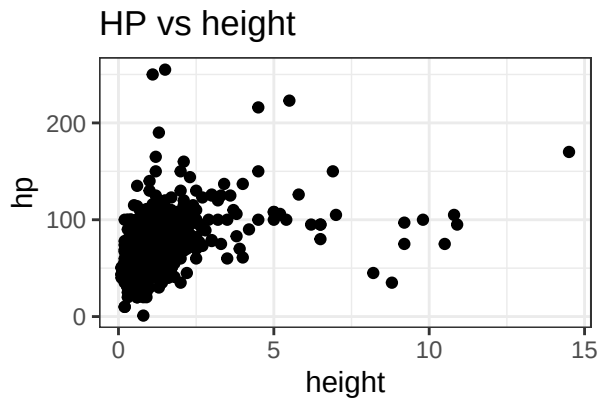
type_panel_3



5) Size + HP (raw vs log)

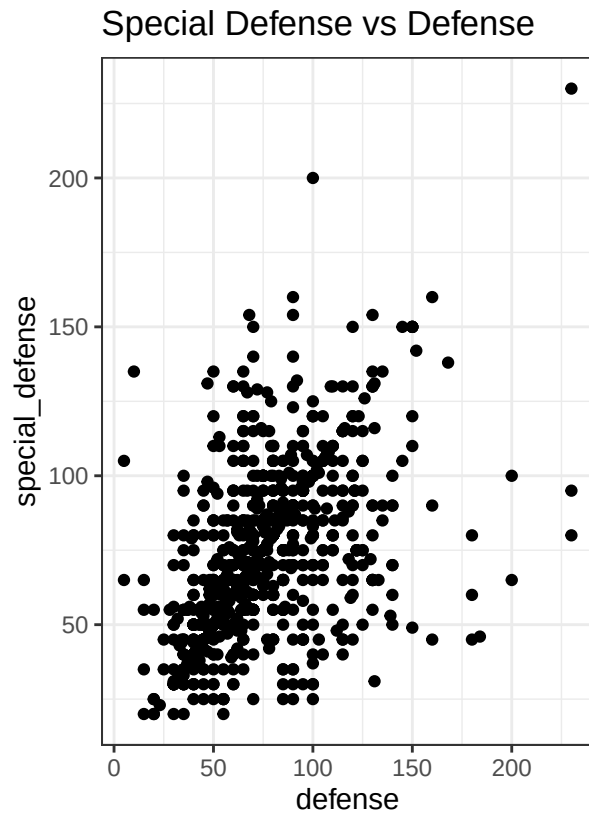
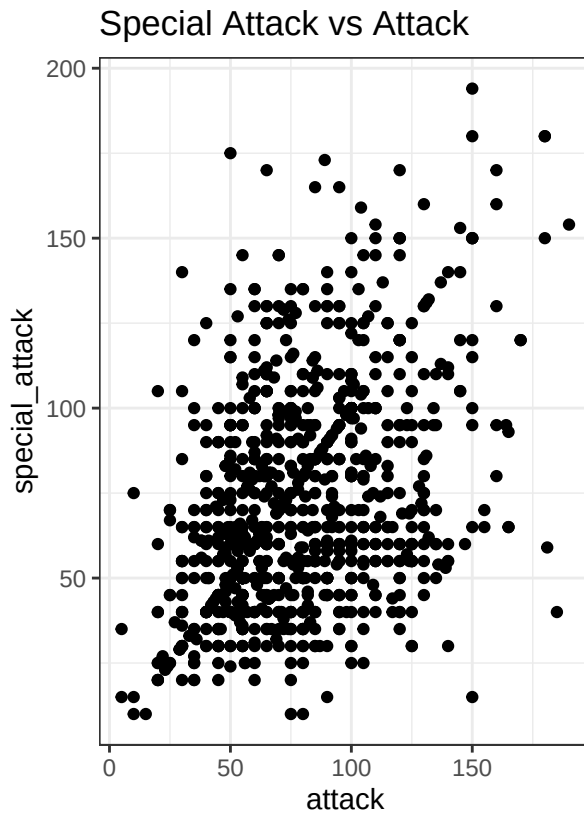
```
p_hp_h <- ggplot(pokemon, aes(height, hp)) + geom_point() + theme_bw() + labs(title="HP vs height")
p_hp_lh <- ggplot(pokemon, aes(log(height), hp)) + geom_point() + theme_bw() + labs(title="HP vs log(height)")
p_hp_w <- ggplot(pokemon, aes(weight, hp)) + geom_point() + theme_bw() + labs(title="HP vs weight")
p_hp_lw <- ggplot(pokemon, aes(log(weight), hp)) + geom_point() + theme_bw() + labs(title="HP vs log(weight)")

hp_panel <- (p_hp_h + p_hp_lh) /
             (p_hp_w + p_hp_lw)
hp_panel
```



6) Variable Relations

```
p_atk_spa <- ggplot(pokemon, aes(attack, special_attack)) + geom_point() + theme_bw() + labs(title="Sp
p_def_spd <- ggplot(pokemon, aes(defense, special_defense)) + geom_point() + theme_bw() + labs(title="Sp
atk_def_panel <- p_atk_spa + p_def_spd
atk_def_panel
```



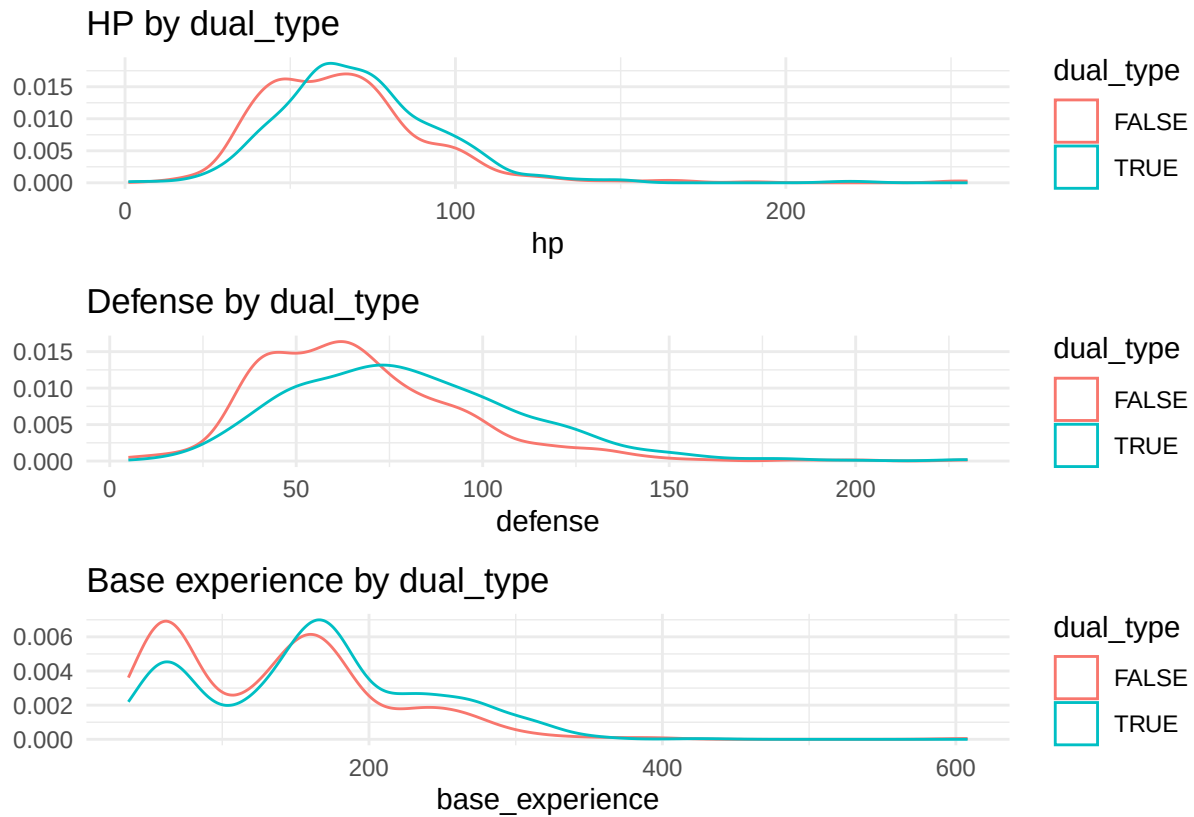
```
eq_counts <- tibble(
  attack_eq_spatk = sum(pokemon$attack == pokemon$special_attack, na.rm = TRUE),
  defense_eq_spdef = sum(pokemon$defense == pokemon$special_defense, na.rm = TRUE),
  both_equal      = sum((pokemon$attack == pokemon$special_attack) &
    (pokemon$defense == pokemon$special_defense), na.rm = TRUE)
)
knitr::kable(eq_counts, digits = 3)
```

attack_eq_spatk	defense_eq_spdef	both_equal
157	321	116

7) Dual-type densities

```
p_den_hp <- ggplot(pokemon, aes(hp, color = dual_type)) + geom_density() + theme_minimal()
p_den_def <- ggplot(pokemon, aes(defense, color = dual_type)) + geom_density() + theme_minimal()
p_den_exp <- ggplot(pokemon, aes(base_experience, color = dual_type)) + geom_density() + theme_minimal()

density_panel <- p_den_hp / p_den_def / p_den_exp
density_panel
```

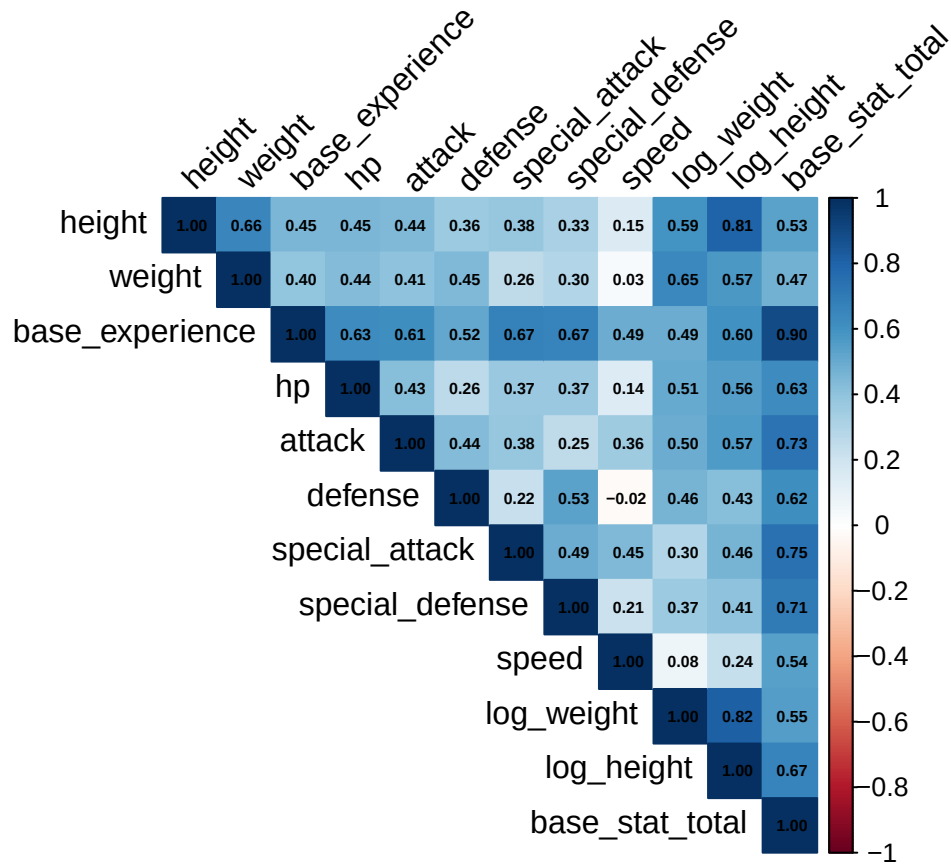


```
dual_means_atk <- pokemon |> group_by(dual_type) |> summarise(`mean(attack)` = mean(attack, na.rm=TRUE))
knitr::kable(dual_means_atk, digits = 3)
```

dual_type	mean(attack)
FALSE	74.415
TRUE	83.825

```
# 8) Correlation heatmap (numeric cols excluding generation_id)
```

```
num_keep <- pokemon |> select(where(is.numeric)) |> select(!generation_id)
M <- cor(num_keep, use = "pairwise.complete.obs")
corrplot::corrplot(M, method = "color", type = "upper", addCoef.col = "black", tl.col = "black",
number.cex = 0.5, tl.srt = 45)
```



```
png("figures/k_corrplot_matrix.png", width = 1200, height = 1200, res = 140)
corrplot::corrplot(M, method = "color", type = "upper", addCoef.col = "black", tl.col = "black",
number.cex = 0.5, tl.srt = 45)
dev.off()
```

```
## pdf
## 2
```

```
# 9) base_stat_total by generation
```

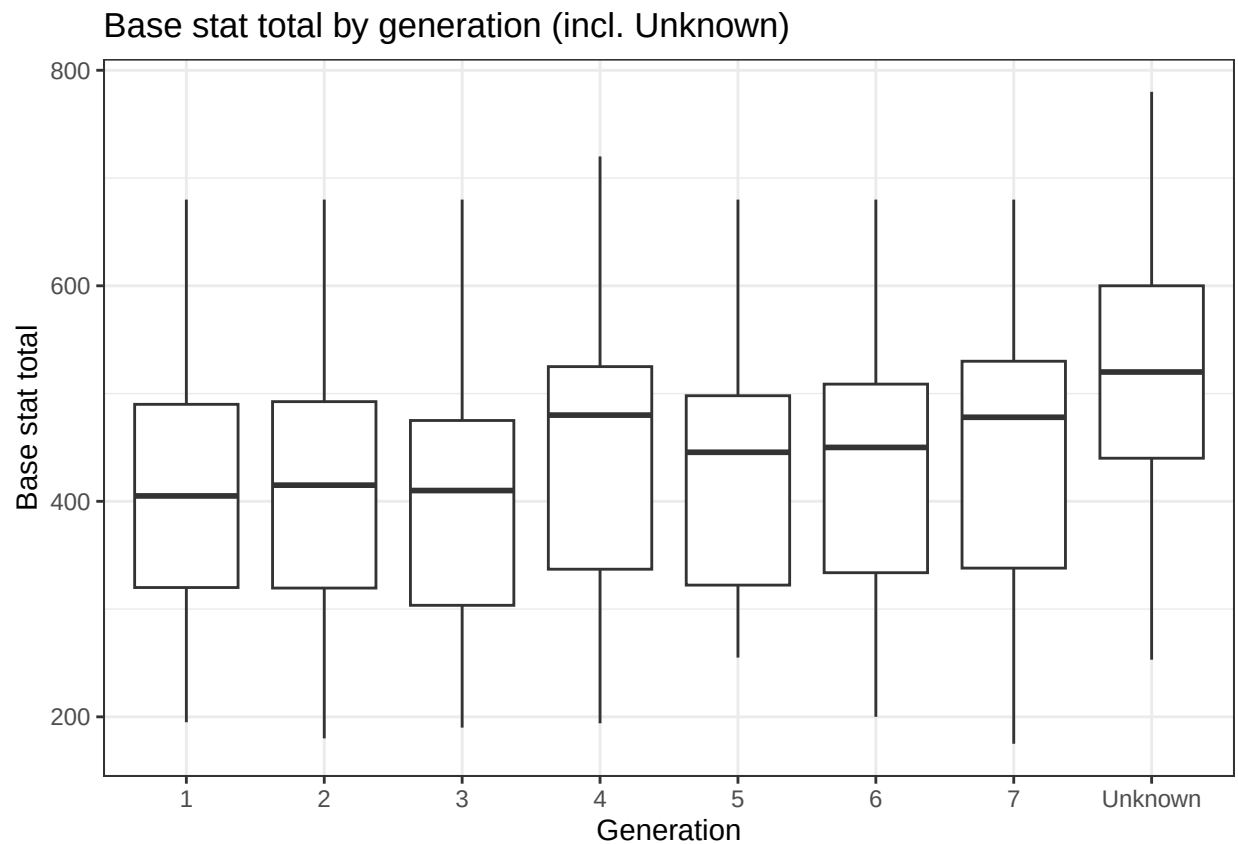
```
gen_total <- pokemon |>
group_by(generation_id_f) |>
summarise(n = n(),
mean_total = mean(base_stat_total, na.rm=TRUE),
sd_total = sd(base_stat_total, na.rm=TRUE)) |>
arrange(generation_id_f)

knitr::kable(gen_total, digits = 1)
```

generation_id_f	n	mean_total	sd_total
1	151	407.6	99.9
2	100	407.2	112.5
3	135	403.7	115.6

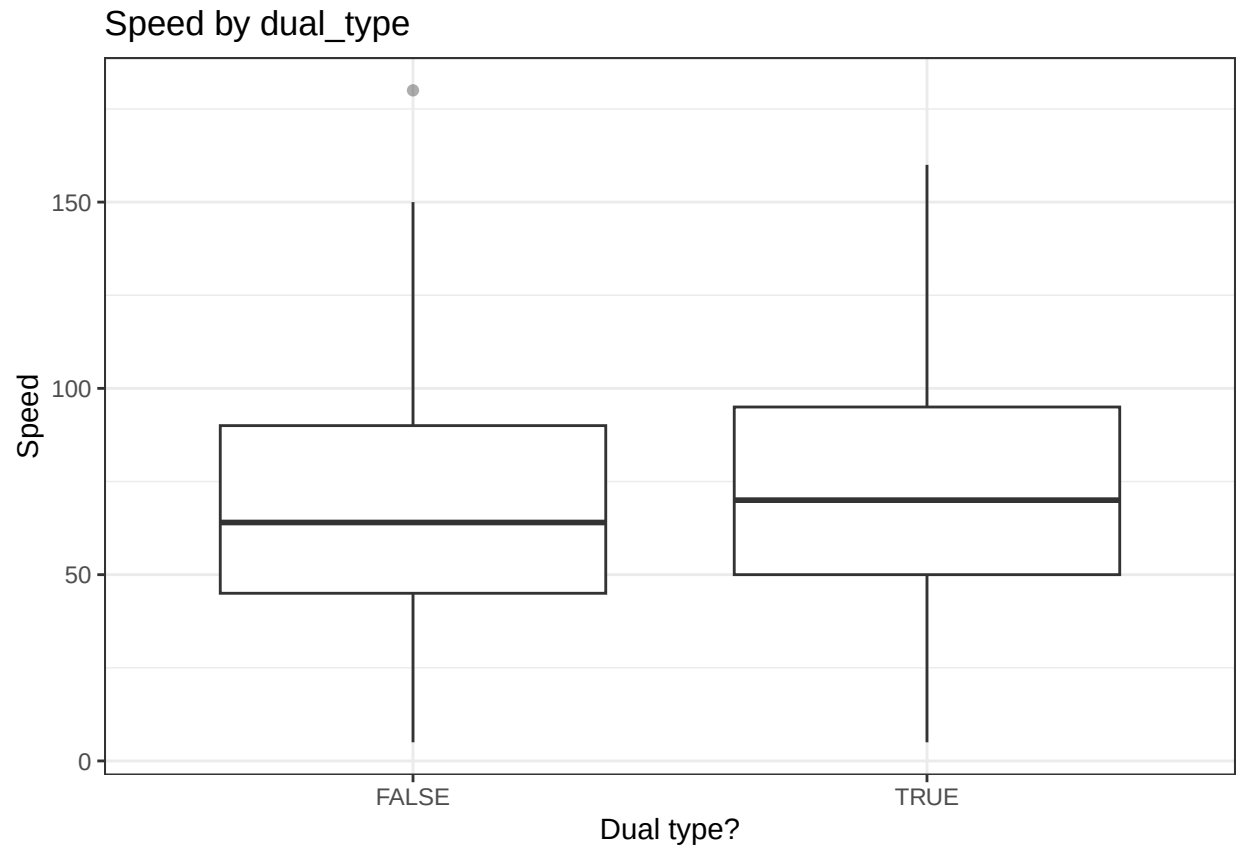
generation_id_f	n	mean_total	sd_total
4	107	445.8	117.5
5	156	425.8	102.5
6	72	429.6	111.9
7	81	443.8	118.7
Unknown	147	520.5	122.9

```
p_gen_total <- ggplot(pokemon, aes(generation_id_f, base_stat_total)) +
  geom_boxplot(outlier.alpha=.4) + theme_bw() +
  labs(title="Base stat total by generation (incl. Unknown)", x="Generation", y="Base stat total")
p_gen_total
```



```
# 10) Speed by dual_type (boxplot + quick t-test)

p_speed_dual <- ggplot(pokemon, aes(x = dual_type, y = speed)) +
  geom_boxplot(outlier.alpha=.4) + theme_bw() +
  labs(title="Speed by dual_type", x="Dual type?", y="Speed")
p_speed_dual
```



```
tt_speed <- broom::tidy(t.test(speed ~ dual_type, data = pokemon))
knitr::kable(tt_speed, digits = 3)
```

estimate	estimate1	estimate2	statistic	p.value	parameter	conf.low	conf.high	method	alternative
-5.163	66.248	71.412	-2.726	0.007	927.629	-8.881	-1.446	Welch Two Sample t-test	two.sided

```
# 11) Base experience VS base stat total (scatter + OLS line)
```

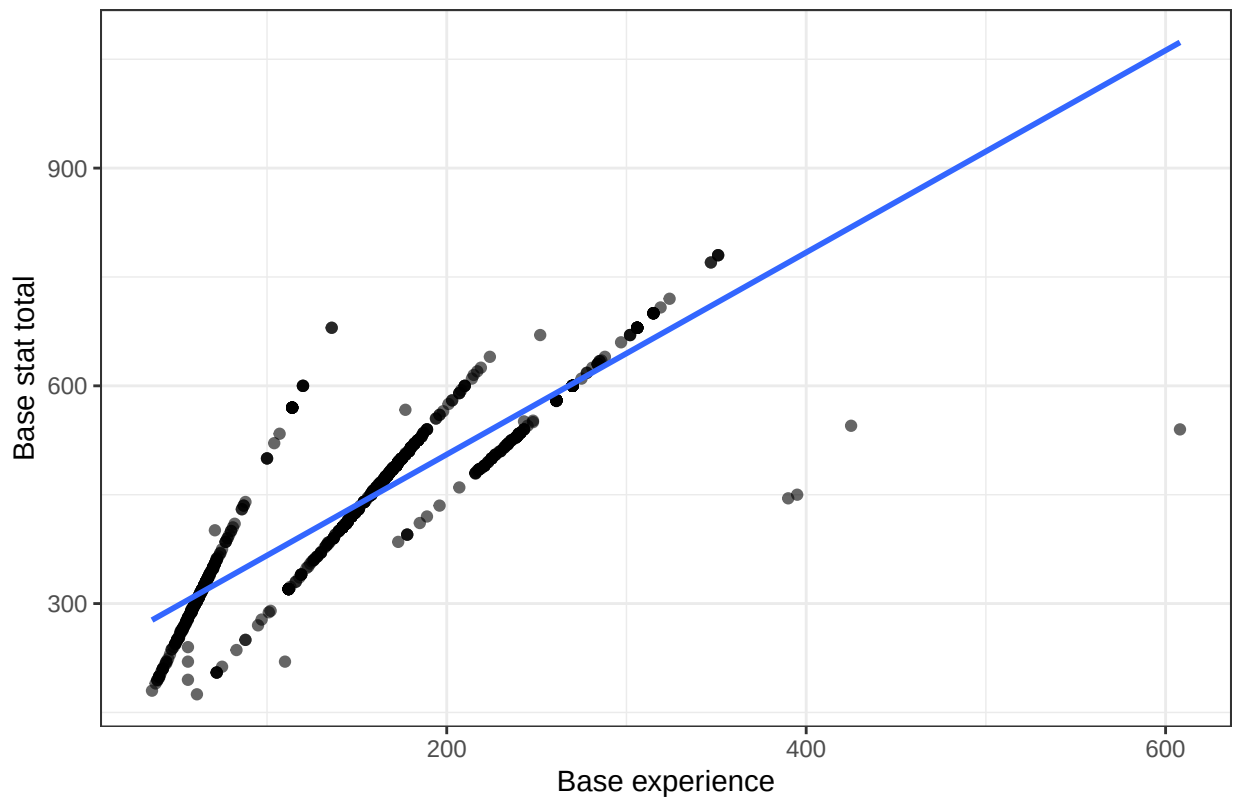
```
p_exp_total <- ggplot(pokemon, aes(base_experience, base_stat_total)) +
  geom_point(alpha=.6) + geom_smooth(method="lm", se=FALSE) + theme_bw() +
  labs(title="Base experience vs base stat total", x="Base experience", y="Base stat total")
ggsave("figures/baseexp_vs_basetotal.png", p_exp_total, width = 7, height = 5, dpi = 150)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
p_exp_total
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Base experience vs base stat total



12) Speed vs log(weight), faceted by top 6 primary types

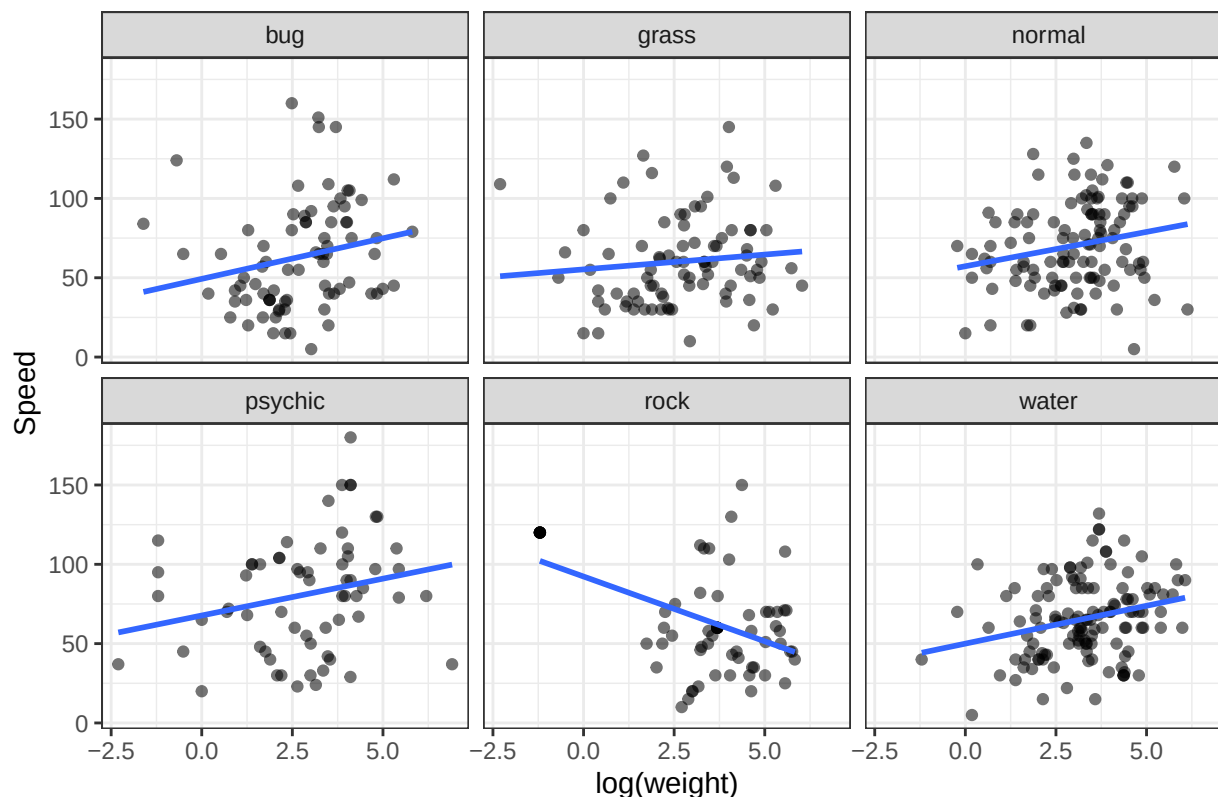
```
top6 <- pokemon |> count(type_1) |> arrange(desc(n)) |> slice_head(n=6) |> pull(type_1)
p_speed_lw_facets <- pokemon |>
  filter(type_1 %in% top6, !is.na(log_weight)) |>
  ggplot(aes(log_weight, speed)) +
  geom_point(alpha=.55) + geom_smooth(method="lm", se=FALSE) +
  facet_wrap(~type_1, ncol=3) + theme_bw() +
  labs(title="Speed vs log(weight): top 6 primary types", x="log(weight)", y="Speed")
ggsave("figures/speed_vs_logweight_top6_types.png", p_speed_lw_facets, width = 9, height = 6, dpi = 150)
```

'geom_smooth()' using formula = 'y ~ x'

p_speed_lw_facets

'geom_smooth()' using formula = 'y ~ x'

Speed vs log(weight): top 6 primary types



Null Hypotheses

After doing EDA, we determined some potential null hypotheses that are worth further exploring.

$$H_0^{(1)} : \beta_{\text{speed}} = \beta_{\text{attack}} = 0, \quad H_0^{(2)} : \beta_{\text{dual_type_x_defense}} = 0, \quad H_0^{(3)} : \beta_{\text{base_experience}} = 0$$

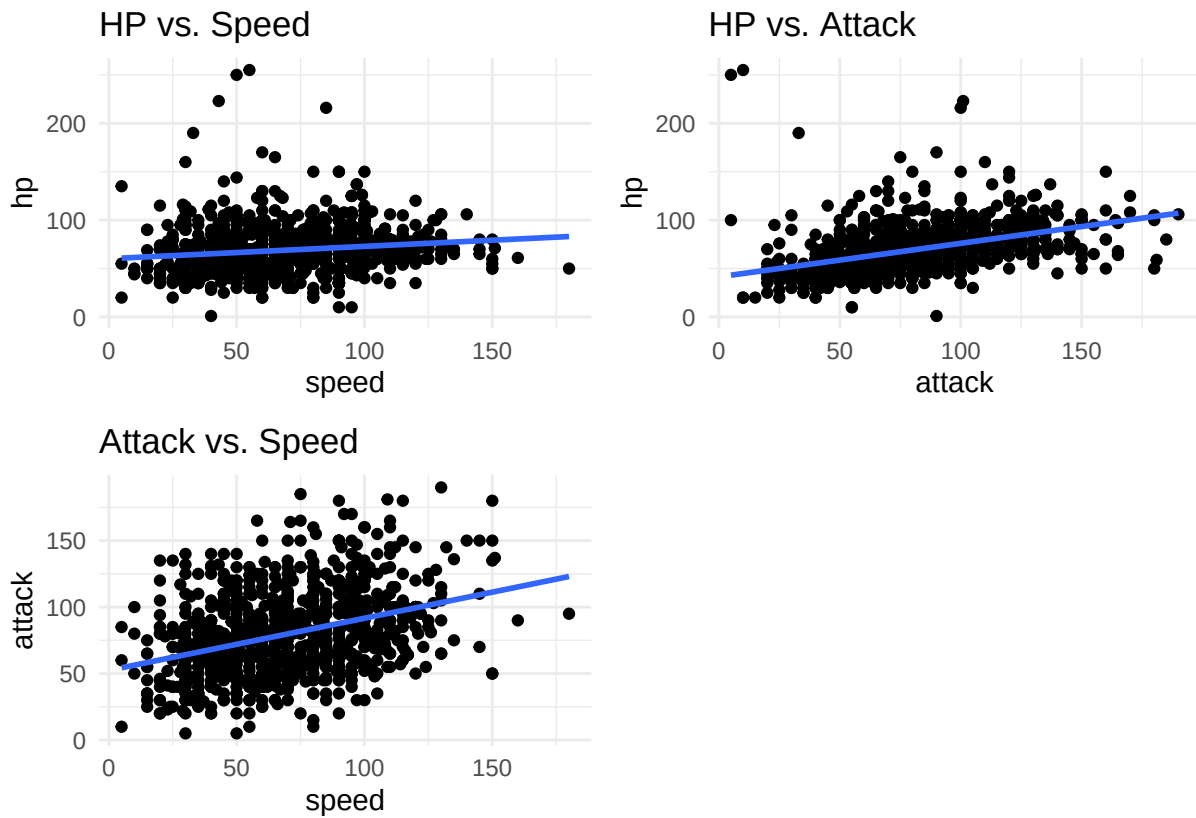
Plots Motivating our Hypothesis

```
#1.
p1 <- ggplot(pokemon, aes(speed, hp))      + geom_point() + geom_smooth(method='lm', se=FALSE) + theme_m
p2 <- ggplot(pokemon, aes(attack, hp))     + geom_point() + geom_smooth(method='lm', se=FALSE) + theme_m
p3 <- ggplot(pokemon, aes(speed, attack))  + geom_point() + geom_smooth(method='lm', se=FALSE) + theme_m
g_tritych <- p1 + p2 + p3 + patchwork::plot_layout(ncol=2)
ggsave("figures/k_hypothesis_tritych.png", plot = g_tritych, width = 9, height = 6, dpi = 150)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
```

```
g_tritych
```

```
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
```

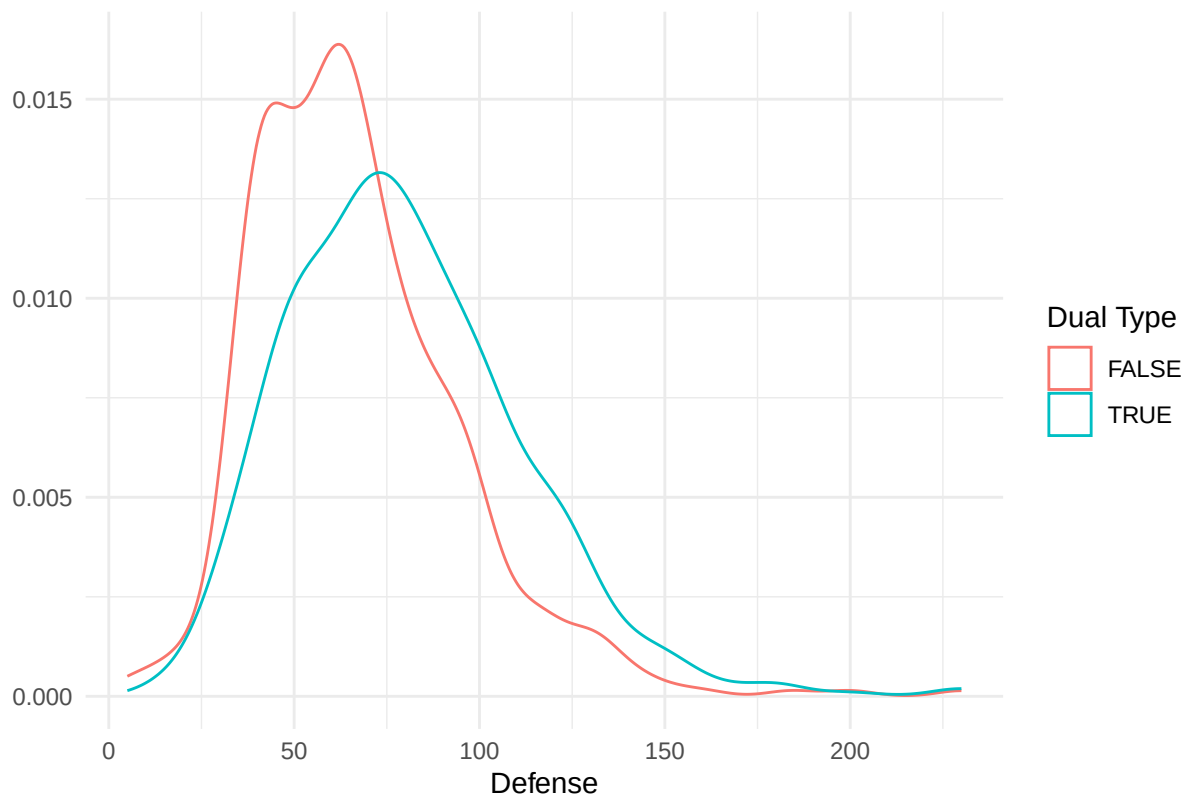


```
corrs_simple <- pokemon |>
  summarise(`cor(speed, hp)` = cor(speed, hp),
    `cor(speed, attack)` = cor(speed, attack),
    `cor(attack, hp)` = cor(attack, hp))
knitr::kable(corrs_simple, digits = 3)
```

cor(speed, hp)	cor(speed, attack)	cor(attack, hp)
0.143	0.359	0.427

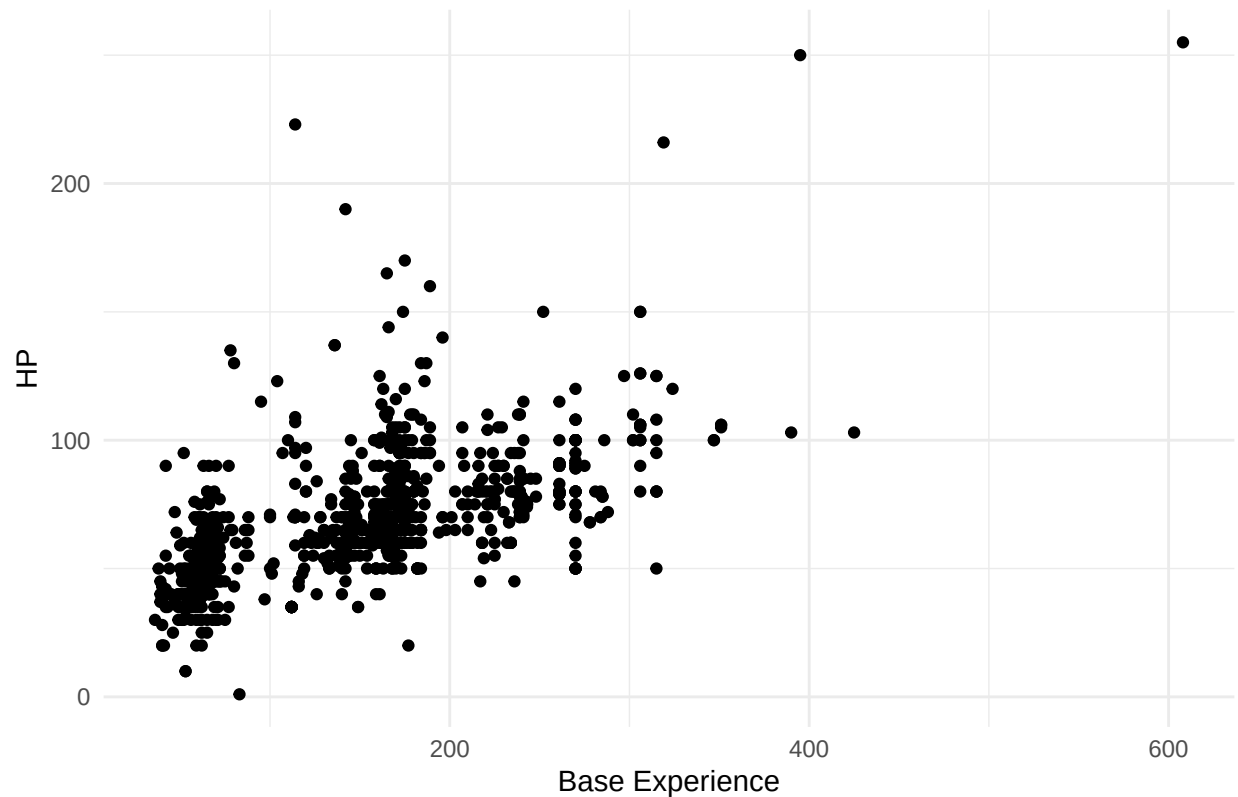
```
#2
ggplot(pokemon) +
  geom_density(aes(x = defense, color = dual_type)) +
  theme_minimal() +
  labs(title = "Distribution of Defense Score by Dual Type",
    color = "Dual Type",
    x = "Defense",
    y = "")
```

Distribution of Defense Score by Dual Type



```
#3
ggplot(pokemon) +
  geom_point(aes(x = base_experience, y = hp)) +
  labs(title = "HP vs. Base Experience",
       x = "Base Experience",
       y = "HP") +
  theme_minimal()
```

HP vs. Base Experience

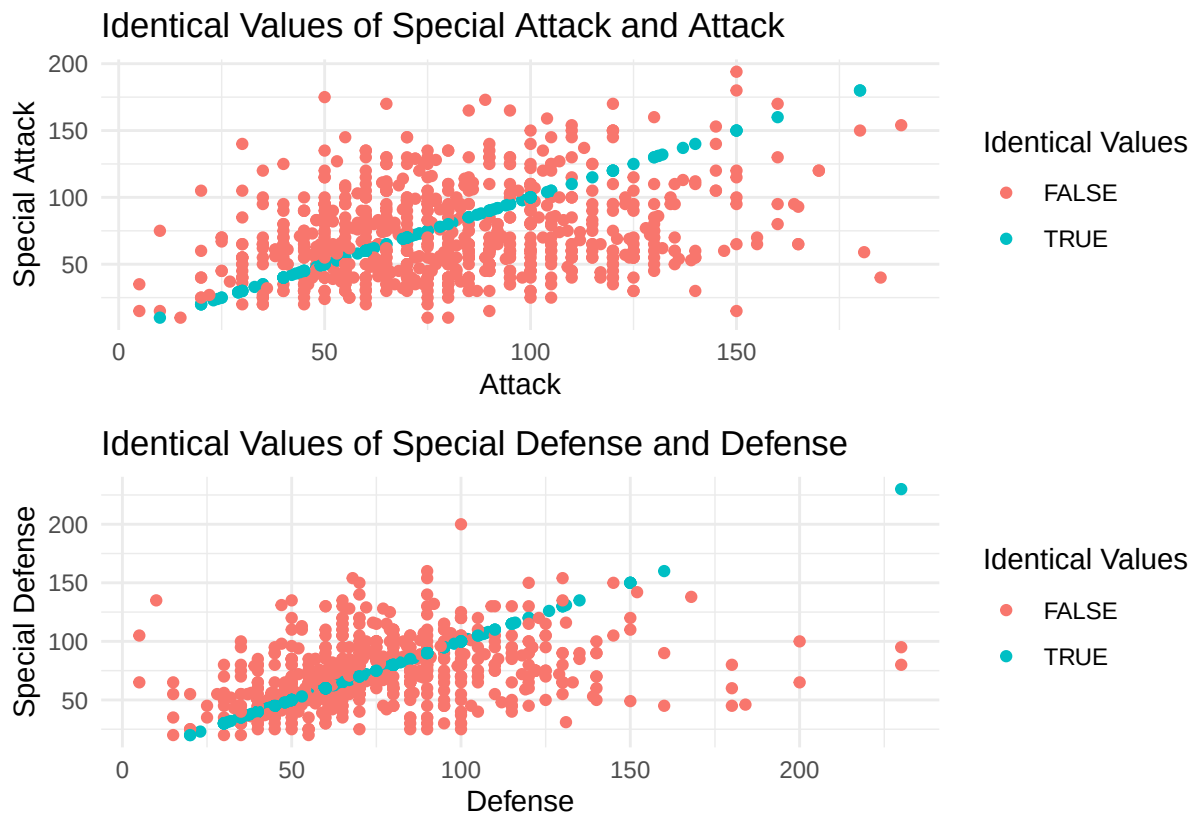


```
#special attack/defense motivation
special_df <- pokemon |>
  mutate(
    same_attack = special_attack == attack,
    same_defense = special_defense == defense
  )

p_same_atk <- ggplot(special_df) +
  geom_point(aes(x = attack, y = special_attack, color = same_attack)) +
  labs(
    title = "Identical Values of Special Attack and Attack",
    x = "Attack",
    y = "Special Attack",
    color = "Identical Values"
  ) +
  theme_minimal()

p_same_def <- ggplot(special_df) +
  geom_point(aes(x = defense, y = special_defense, color = same_defense)) +
  labs(
    title = "Identical Values of Special Defense and Defense",
    x = "Defense",
    y = "Special Defense",
    color = "Identical Values"
  ) +
  theme_minimal()
```

```
same_panel <- p_same_atk / p_same_def
same_panel
```



Limitations of dataset

This dataset doesn't include some characteristics of Pokemon that may or may not be helpful in answering our questions. For example, it doesn't include the catch rate of Pokemon or if the Pokemon is legendary/mythical or not.

Proposed analysis plan

The team will carry out an iterative, back-and-forth model-building process within the linear regression framework for predicting HP, trying different combinations of predictors and interactions. At each stage, we will use methods discussed in class—primarily *t-tests* for individual coefficients and *F-tests* for groups of terms—to assess which variables and interactions meaningfully contribute to the model. Once we arrive at a well-supported specification, we will use the fitted model to test the three hypotheses outlined earlier, reject or fail to reject them based on the results, and summarize our conclusions in the final project write-up.

Acknowledgements

ChatGPT was used to help with coding. Yin and Kareena did the EDA and made the figures. Irene did the written portion of the report. Sam made the presentation.